Model Evaluation Metrics

Classification and Regression Quick Reference

Choose the metric that matches the cost of mistakes.

by Bhuvnesh Verma

github.com/MasterBhuvnesh/notebooks

Contents

1. Evaluation Basics

Good metrics are decision tools. A model can look strong on one metric and still fail in production. The best metric is the one that reflects the real business cost of being wrong.

Question	What to look for
Is the target imbalanced?	Accuracy is often misleading. Check precision, recall, F1, or PR-AUC.
Are false negatives costly?	Focus on recall and sensitivity.
Are false positives costly?	Focus on precision and specificity.
Are predictions continuous?	Use MAE, MSE, RMSE, or R^2.
Do you need ranking quality?	Use ROC curve or PR curve.

2. Classification Metrics

2.1 Confusion Matrix

What is it? A confusion matrix counts correct and incorrect predictions by class. It is the clearest way to see where a classifier fails.

Used For: Classification

When to use: Diagnosing error types, checking class imbalance, fixing recall or precision issues, and understanding the model's mistakes.

Formula: For binary classification, the matrix is

$$\begin{bmatrix} TP & FP \\ FN & TN \end{bmatrix}$$

where each cell is a count of predictions.

Symbols: TP = true positives, FP = false positives, FN = false negatives, TN = true negatives.

What Does It Measure? It measures the distribution of predictions across the actual classes. It tells you whether the model is confusing one class for another.

Interpretation:

Value	Meaning
Large diagonal, small off-diagonal	Good separation between classes.
Many FN	The model misses positive cases.
Many FP	The model is too eager to label positives.

Example: Suppose 100 patients are tested for a rare disease. Results are: $TP = 24$, $FN = 6$, $FP = 8$, $TN = 62$. The confusion matrix is:

$$\begin{bmatrix} 24 & 8 \\ 6 & 62 \end{bmatrix}$$

This means 24 diseased patients were correctly detected, 6 were missed, 8 healthy patients were wrongly flagged, and 62 healthy patients were correctly rejected.

Python (Using Library):

```python
from sklearn.metrics import confusion_matrix

y_true = [1, 0, 1, 1, 0, 0]
y_pred = [1, 0, 0, 1, 0, 1]

cm = confusion_matrix(y_true, y_pred)
print(cm)
# [[2 1]
#  [1 2]]
```

Python (Without Library):

```python
def confusion_matrix_py(y_true, y_pred):
    tp = fp = fn = tn = 0
    for yt, yp in zip(y_true, y_pred):
        if yt == 1 and yp == 1:
            tp += 1
        elif yt == 0 and yp == 1:
            fp += 1
        elif yt == 1 and yp == 0:
            fn += 1
        else:
            tn += 1
    return [[tp, fp], [fn, tn]]
```

Quick Notes:

- Advantages: shows exact error pattern; useful for class imbalance and threshold tuning.
- Limitations: not a single score; can be hard to compare across models at a glance.
- Best use cases: binary classification, medical screening, fraud detection, churn prediction.

2.2 Accuracy

What is it? Accuracy is the share of all predictions that are correct.

Used For: Classification

When to use: Balanced datasets with roughly equal class frequencies and similar error costs.

Formula:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Symbols: TP, TN, FP, FN are the confusion-matrix counts.

What Does It Measure? It measures overall correctness, but it does not tell you whether the model is failing on the important class.

Interpretation:

Value	Meaning
Near 1.0	Very accurate overall.
Near 0.5	Barely better than random guessing.
Lower than expected on imbalanced data	High accuracy may hide poor minority-class performance.

Example: Using the previous confusion matrix, accuracy is

$$\frac{24 + 62}{24 + 62 + 8 + 6} = \frac{86}{100} = 0.86$$

So the model is 86% accurate overall.

Python (Using Library):

```python
from sklearn.metrics import accuracy_score

y_true = [1, 0, 1, 1, 0, 0]
y_pred = [1, 0, 0, 1, 0, 1]
print(accuracy_score(y_true, y_pred))
# 0.6666666666666666
```

Python (Without Library):

```python
def accuracy_score_py(y_true, y_pred):
    correct = sum(1 for yt, yp in zip(y_true, y_pred) if yt == yp)
    return correct / len(y_true)
```

Quick Notes:

- Advantages: easy to understand and communicate.
- Limitations: can be misleading on imbalanced data.
- Best use cases: balanced classification tasks and baseline comparisons.

2.3 Precision

What is it? Precision asks: of the instances predicted as positive, how many were actually positive?

Used For: Classification

When to use: When false positives are costly, such as email spam detection or fraud alerts.

Formula:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Symbols: TP = true positives, FP = false positives.

What Does It Measure? It measures the quality of the positive predictions. A high precision means the model is not labelling too many negatives as positives.

Interpretation:

Value	Meaning
Closer to 1.0	More reliable positive predictions.
Closer to 0.0	Many false alarms.

Example: Using the confusion matrix above,

$$\text{Precision} = \frac{24}{24 + 8} = \frac{24}{32} = 0.75$$

So 75% of predicted positive cases were truly positive.

Python (Using Library):

```python
from sklearn.metrics import precision_score

y_true = [1, 0, 1, 1, 0, 0]
y_pred = [1, 0, 0, 1, 0, 1]
print(precision_score(y_true, y_pred))
# 0.6666666666666666
```

Python (Without Library):

```python
def precision_score_py(y_true, y_pred):
    tp = sum(1 for yt, yp in zip(y_true, y_pred) if yt == 1 and yp == 1)
    fp = sum(1 for yt, yp in zip(y_true, y_pred) if yt == 0 and yp == 1)
    return tp / (tp + fp) if (tp + fp) else 0.0
```

Quick Notes:

- Advantages: useful when false positives are expensive.
- Limitations: ignores false negatives.
- Best use cases: fraud detection, email filtering, spam detection.

2.4 Recall

What is it? Recall asks: of the actual positive cases, how many did the model find?

Used For: Classification

When to use: When missing positive cases is costly, such as disease detection or churn prevention.

Formula:

$$\text{Recall} = \frac{TP}{TP + FN}$$

Symbols: TP = true positives, FN = false negatives.

What Does It Measure? It measures sensitivity to the positive class. A high recall means the model catches most real positives.

Interpretation:

Value	Meaning
Closer to 1.0	Few positives are missed.
Closer to 0.0	Many real positives are being missed.

Example: With the same confusion matrix,

$$\text{Recall} = \frac{24}{24 + 6} = \frac{24}{30} = 0.80$$

The model finds 80% of actual positive cases.

Python (Using Library):

```python
from sklearn.metrics import recall_score

y_true = [1, 0, 1, 1, 0, 0]
y_pred = [1, 0, 0, 1, 0, 1]
print(recall_score(y_true, y_pred))
# 0.8
```

Python (Without Library):

```python
def recall_score_py(y_true, y_pred):
    tp = sum(1 for yt, yp in zip(y_true, y_pred) if yt == 1 and yp == 1)
    fn = sum(1 for yt, yp in zip(y_true, y_pred) if yt == 1 and yp == 0)
    return tp / (tp + fn) if (tp + fn) else 0.0
```

Quick Notes:

- Advantages: captures missed positive cases.
- Limitations: ignores false positives.
- Best use cases: disease detection, anomaly detection, retention outreach.

2.5 F1 Score

What is it? The F1 score is the harmonic mean of precision and recall. It balances both in one number.

Used For: Classification

When to use: Class imbalance, where accuracy is not informative and both false alarms and missed positives matter.

Formula:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Symbols: Precision = fraction of predicted positives that are correct; Recall = fraction of actual positives that are found.

What Does It Measure? It measures the trade-off between precision and recall in a single score. It punishes weak performance on either side.

Interpretation:

Value	Meaning
1.0	Perfect precision and recall.
0.0	No positive predictions are correct.
Higher is better	Better balance between catching positives and avoiding false alarms.

Example: Using precision = 0.75 and recall = 0.80,

$$F_1 = 2 \cdot \frac{0.75 \cdot 0.80}{0.75 + 0.80} = 2 \cdot \frac{0.60}{1.55} \approx 0.774$$

The model's F1 score is about 0.77.

Python (Using Library):

```python
from sklearn.metrics import f1_score

y_true = [1, 0, 1, 1, 0, 0]
y_pred = [1, 0, 0, 1, 0, 1]
print(f1_score(y_true, y_pred))
# 0.6666666666666666
```

Python (Without Library):

```python
def f1_score_py(y_true, y_pred):
    tp = sum(1 for yt, yp in zip(y_true, y_pred) if yt == 1 and yp == 1)
    fp = sum(1 for yt, yp in zip(y_true, y_pred) if yt == 0 and yp == 1)
    fn = sum(1 for yt, yp in zip(y_true, y_pred) if yt == 1 and yp == 0)
    precision = tp / (tp + fp) if (tp + fp) else 0.0
    recall = tp / (tp + fn) if (tp + fn) else 0.0
    return 2 * precision * recall / (precision + recall) if (precision + recall) else 0.0
```

Quick Notes:

- Advantages: one-number summary of precision-recall trade-off.
- Limitations: depends on the chosen positive class and threshold.
- Best use cases: imbalanced classification, anomaly detection, rare-event prediction.

2.6 Specificity

What is it? Specificity measures how well the model identifies negative cases. It is also called the true negative rate.

Used For: Classification

When to use: When avoiding false alarms matters as much as capturing positives.

Formula:

$$\text{Specificity} = \frac{TN}{TN + FP}$$

Symbols: TN = true negatives, FP = false positives.

What Does It Measure? It tells you how often the model correctly says "no" for the negative class.

Interpretation:

Value	Meaning
Closer to 1.0	Very good at ruling out negatives.
Closer to 0.0	Many healthy cases are incorrectly flagged.

Example: With the matrix above,

$$\text{Specificity} = \frac{62}{62+8} = \frac{62}{70} = 0.886$$

So the model correctly rejects 88.6% of healthy cases.

Python (Using Library):

```python
from sklearn.metrics import confusion_matrix

y_true = [1, 0, 1, 1, 0, 0]
y_pred = [1, 0, 0, 1, 0, 1]
cm = confusion_matrix(y_true, y_pred)
tn, fp, fn, tp = cm.ravel()
print((tn / (tn + fp)))
# 0.5
```

Python (Without Library):

```python
def specificity_score_py(y_true, y_pred):
    tn = sum(1 for yt, yp in zip(y_true, y_pred) if yt == 0 and yp == 0)
    fp = sum(1 for yt, yp in zip(y_true, y_pred) if yt == 0 and yp == 1)
    return tn / (tn + fp) if (tn + fp) else 0.0
```

Quick Notes:

- Advantages: useful when false positives are expensive.
- Limitations: less informative alone; it should be read alongside recall or sensitivity.
- Best use cases: medical screening, quality control, safety systems.

2.7 ROC Curve

What is it? The ROC curve plots true positive rate against false positive rate at different decision thresholds.

Used For: Classification

When to use: Comparing classifiers across thresholds, especially when positive class is rare but ranking matters.

Formula:

$$TPR = \frac{TP}{TP+FN}, \qquad FPR = \frac{FP}{FP+TN}$$

The ROC curve is the plot of TPR against FPR over different thresholds.

Symbols: TPR = true positive rate, also recall; FPR = false positive rate, the share of negatives wrongly predicted as positive.

What Does It Measure? It measures the model's ability to rank positive cases above negative cases as the threshold changes.

Interpretation:

Value	Meaning
Curve near upper-left corner	Strong discrimination.
Diagonal line	Random guessing.
Lower curve	Worse ranking ability.

Example: If a model is very good at ranking positives ahead of negatives, its ROC curve will rise sharply near the top-left, giving high true-positive rate while keeping false-positive rate low.

Python (Using Library):

```python
from sklearn.metrics import roc_curve
import numpy as np

proba = np.array([0.95, 0.80, 0.40, 0.30, 0.70, 0.20])
y_true = np.array([1, 1, 0, 0, 1, 0])

fpr, tpr, thresholds = roc_curve(y_true, proba)
print(fpr)
print(tpr)
```

Python (Without Library):

```python
def roc_curve_py(y_true, scores):
    thresholds = sorted(set(scores), reverse=True)
    points = []
    for t in thresholds:
        pred = [1 if s >= t else 0 for s in scores]
        tp = sum(1 for yt, yp in zip(y_true, pred) if yt == 1 and yp == 1)
        fp = sum(1 for yt, yp in zip(y_true, pred) if yt == 0 and yp == 1)
        fn = sum(1 for yt, yp in zip(y_true, pred) if yt == 1 and yp == 0)
        tn = sum(1 for yt, yp in zip(y_true, pred) if yt == 0 and yp == 0)
        tpr = tp / (tp + fn) if (tp + fn) else 0.0
        fpr = fp / (fp + tn) if (fp + tn) else 0.0
        points.append((fpr, tpr, t))
    return points
```

Quick Notes:

- Advantages: threshold-independent view of ranking quality.
- Limitations: less informative when class imbalance is extreme and the operational threshold is fixed.
- Best use cases: comparing candidate models, ranking systems, probability calibration checks.

2.8 AUC

What is it? AUC is the area under the ROC curve. It summarizes the ROC curve into a single number.

Used For: Classification

When to use: Comparing the overall ranking power of models when the decision threshold is not fixed.

Formula:

$$\text{AUC} = \int_0^1 TPR(FPR)\, d(FPR)$$

Symbols: TPR = true positive rate, FPR = false positive rate.

What Does It Measure? It measures how often the model ranks a random positive example above a random negative example.

Interpretation:

Value	Meaning
1.0	Perfect ranking.
0.5	Random guessing.
Below 0.5	Worse than random.
Higher is better	Better model discrimination.

Example: If the ROC curve is close to the upper-left corner, the AUC may be 0.90 or 0.95. That means the model is often correct when ordering a random positive versus a random negative case.

Python (Using Library):

```
from sklearn.metrics import roc_auc_score
import numpy as np

proba = np.array([0.95, 0.80, 0.40, 0.30, 0.70, 0.20])
y_true = np.array([1, 1, 0, 0, 1, 0])
print(roc_auc_score(y_true, proba))
```

Python (Without Library):

```
def auc_score_py(y_true, scores):
    pos = [s for yt, s in zip(y_true, scores) if yt == 1]
    neg = [s for yt, s in zip(y_true, scores) if yt == 0]
    total = 0
    for p in pos:
        for n in neg:
            total += 1 if p > n else 0.5 if p == n else 0
    return total / (len(pos) * len(neg)) if pos and neg else 0.0
```

Quick Notes:

- Advantages: single scalar summary of ranking quality; threshold-independent.
- Limitations: does not show where the operating threshold lies.
- Best use cases: credit scoring, fraud models, recommendation ranking, binary classification benchmarks.

2.9 Precision-Recall Curve

What is it? The precision-recall curve shows how precision and recall trade off as the decision threshold changes.

Used For: Classification

When to use: Class imbalance or when the positive class is rare and precision matters a lot.

Formula:

$$\text{Precision}(t) = \frac{TP(t)}{TP(t) + FP(t)}, \qquad \text{Recall}(t) = \frac{TP(t)}{TP(t) + FN(t)}$$

with threshold t varying across the score range.

Symbols: t = decision threshold.

What Does It Measure? It measures whether the model can keep precision high while still finding most positive cases.

Interpretation:

Value	Meaning
Curve stays high	Good trade-off between precision and recall.
Sharp drop in precision at high recall	Too many false alarms when recall rises.
Higher area under the curve	Better performance on rare positives.

Example: In fraud detection, you may accept lower recall if precision is critical. The PR curve lets you inspect that trade-off directly.

Python (Using Library):

```python
from sklearn.metrics import precision_recall_curve
import numpy as np

proba = np.array([0.95, 0.90, 0.70, 0.30, 0.60, 0.10])
y_true = np.array([1, 1, 0, 0, 1, 0])
precision, recall, thresholds = precision_recall_curve(y_true, proba)
print(precision)
print(recall)
```

Python (Without Library):

```python
def precision_recall_curve_py(y_true, scores):
    thresholds = sorted(set(scores), reverse=True)
    results = []
    for t in thresholds:
        pred = [1 if s >= t else 0 for s in scores]
        tp = sum(1 for yt, yp in zip(y_true, pred) if yt == 1 and yp == 1)
        fp = sum(1 for yt, yp in zip(y_true, pred) if yt == 0 and yp == 1)
        fn = sum(1 for yt, yp in zip(y_true, pred) if yt == 1 and yp == 0)
        precision = tp / (tp + fp) if (tp + fp) else 0.0
        recall = tp / (tp + fn) if (tp + fn) else 0.0
        results.append((t, precision, recall))
    return results
```

Quick Notes:

- Advantages: very informative for rare positives and class imbalance.
- Limitations: less standard than ROC in some business settings.
- Best use cases: fraud, default prediction, disease detection, anomaly detection.

2.10 Log Loss

What is it? Log loss measures how close predicted probabilities are to the true class labels. It is a probabilistic loss function.

Used For: Classification

When to use: Comparing probabilistic predictions, especially in binary or multiclass classification.

Formula:

$$\text{LogLoss} = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log(p_i) + (1-y_i)\log(1-p_i)]$$

Symbols: n = number of examples; $y_i \in \{0,1\}$ = true label; p_i = predicted probability of class 1.

What Does It Measure? It penalizes confident wrong predictions more heavily than modest mistakes. The model is rewarded for probabilities close to the truth.

Interpretation:

Value	Meaning
Lower is better	Better probability calibration and predictions.
Very high values	The model is confident but wrong.

Example: Suppose the true label is 1 and the model predicts $p = 0.9$. Then the loss is

$$-\log(0.9) \approx 0.105$$

If the model predicts $p = 0.2$, the loss is

$$-\log(0.2) \approx 1.609$$

The second prediction is much worse, even though it is still a class prediction.

Python (Using Library):

```
from sklearn.metrics import log_loss

proba = [0.9, 0.2, 0.8, 0.3]
y_true = [1, 0, 1, 0]
print(log_loss(y_true, proba))
```

Python (Without Library):

```python
import math

def log_loss_py(y_true, probs):
    total = 0.0
    for y, p in zip(y_true, probs):
        p = min(max(p, 1e-15), 1 - 1e-15)
        total += -(y * math.log(p) + (1 - y) * math.log(1 - p))
    return total / len(y_true)
```

Quick Notes:

- Advantages: evaluates probability quality, not just hard labels; sensitive to confident mistakes.
- Limitations: not directly interpretable as accuracy or business impact; harder to explain to non-technical stakeholders.
- Best use cases: model comparison, probability predictions, ranking and calibration tasks.

3. Regression Metrics

3.1 MAE (Mean Absolute Error)

What is it? MAE measures the average absolute difference between predicted and actual values.

Used For: Regression

When to use: When you want an easy-to-explain average error in the same units as the target.

Formula:

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

Symbols: n = number of samples; y_i = actual value; $\hat{y}_i$ = predicted value.

What Does It Measure? It measures how far predictions are from the truth, on average, without squaring large errors.

Interpretation:

Value	Meaning
Lower is better	Predictions are closer to the actual values.

Example: Actual values: $[10, 12, 14]$; predictions: $[12, 11, 13]$.

$$|10-12| + |12-11| + |14-13| = 2 + 1 + 1 = 4$$

$$\text{MAE} = \frac{4}{3} \approx 1.33$$

The model misses by about 1.33 units on average.

Python (Using Library):

```python
from sklearn.metrics import mean_absolute_error

y_true = [10, 12, 14]
y_pred = [12, 11, 13]
print(mean_absolute_error(y_true, y_pred))
# 1.3333333333333333
```

Python (Without Library):

```python
def mae_py(y_true, y_pred):
    return sum(abs(y - p) for y, p in zip(y_true, y_pred)) / len(y_true)
```

Quick Notes:

- Advantages: easy to interpret; robust to outliers compared with MSE.
- Limitations: does not penalize large errors as strongly as squared-error metrics.
- Best use cases: sales forecasting, household energy prediction, average error reporting.

3.2 MSE (Mean Squared Error)

What is it? MSE is the average of squared prediction errors.

Used For: Regression

When to use: When larger errors should be punished more strongly than smaller ones.

Formula:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Symbols: y_i = actual value, $\hat{y}_i$ = predicted value.

What Does It Measure? It measures the average squared distance between predictions and actual values. Squaring emphasizes large mistakes.

Interpretation:

Value	Meaning
Lower is better	Predictions are closer overall.
Large values	Big errors are dominating the score.

Example: Using the same values,

$$(10-12)^2 + (12-11)^2 + (14-13)^2 = 4 + 1 + 1 = 6$$

$$\text{MSE} = \frac{6}{3} = 2$$

The model has squared error of 2.

Python (Using Library):

```python
from sklearn.metrics import mean_squared_error

y_true = [10, 12, 14]
y_pred = [12, 11, 13]
print(mean_squared_error(y_true, y_pred))
# 2.0
```

Python (Without Library):

```python
def mse_py(y_true, y_pred):
    return sum((y - p) ** 2 for y, p in zip(y_true, y_pred)) / len(y_true)
```

Quick Notes:

- Advantages: penalizes large errors strongly; widely used in optimization.
- Limitations: units are squared, so harder to interpret directly.
- Best use cases: optimization objectives, regression benchmarking, loss functions.

3.3 RMSE (Root Mean Squared Error)

What is it? RMSE is the square root of MSE, bringing the error back to the original unit scale.

Used For: Regression

When to use: When you want a metric in the same units as the target while still penalizing large errors.

Formula:

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

Symbols: n = sample count; y_i = actual value; $\hat{y}_i$ = predicted value.

What Does It Measure? It measures the typical size of the residuals in the original units. It gives more weight to large mistakes than MAE.

Interpretation:

Value	Meaning
Lower is better	Error magnitude is smaller.
Large differences between RMSE and MAE	Large errors are present.

Example: Since MSE is 2,

$$\text{RMSE} = \sqrt{2} \approx 1.41$$

The typical error is about 1.41 units.

Python (Using Library):

```python
from sklearn.metrics import mean_squared_error
import math

y_true = [10, 12, 14]
y_pred = [12, 11, 13]
rmse = math.sqrt(mean_squared_error(y_true, y_pred))
print(rmse)
# 1.4142135623730951
```

Python (Without Library):

```python
import math

def rmse_py(y_true, y_pred):
    mse = sum((y - p) ** 2 for y, p in zip(y_true, y_pred)) / len(y_true)
    return math.sqrt(mse)
```

Quick Notes:

- Advantages: easy to interpret; widely used in forecasting.

- Limitations: still sensitive to outliers.
- Best use cases: demand forecasting, house-price prediction, risk models.

3.4 R^2 Score

What is it? R^2 measures how much better the model is than predicting the mean of the target.

Used For: Regression

When to use: Comparing regression models and checking how much variance is explained by the model.

Formula:

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

Symbols: $\bar{y}$ = mean of actual target values; y_i = actual value; $\hat{y}_i$ = predicted value.

What Does It Measure? It compares the model's residual error to the baseline error of always predicting the mean. Higher values mean the model explains more variance.

Interpretation:

Value	Meaning
1.0	Perfect predictions.
0.0	Model performs like using the mean baseline.
Negative values	Worse than the mean baseline.
Higher is better	Better explanatory power.

Example: Suppose actual values are $[10, 12, 14]$ and predictions are $[12, 11, 13]$. The mean is 12. The numerator is 6 and the denominator is 4. So:

$$R^2 = 1 - \frac{6}{4} = -0.5$$

This is worse than predicting the mean.

Python (Using Library):

```python
from sklearn.metrics import r2_score

y_true = [10, 12, 14]
y_pred = [12, 11, 13]
print(r2_score(y_true, y_pred))
# -0.5
```

Python (Without Library):

```python
def r2_score_py(y_true, y_pred):
    y_mean = sum(y_true) / len(y_true)
    ss_res = sum((y - p) ** 2 for y, p in zip(y_true, y_pred))
    ss_tot = sum((y - y_mean) ** 2 for y in y_true)
    return 1 - (ss_res / ss_tot) if ss_tot else 0.0
```

Quick Notes:

- Advantages: interpretable and common in regression reporting.
- Limitations: can increase when adding features even if they are not useful; does not penalize complexity.
- Best use cases: model comparison, variance explained, regression summaries.

3.5 Adjusted R^2

What is it? Adjusted R^2 modifies R^2 to account for the number of predictors in the model.

Used For: Regression

When to use: Comparing models with different numbers of variables, especially in feature selection.

Formula:

$$\text{Adjusted } R^2 = 1 - \left(\frac{1 - R^2}{n - k - 1}\right)(n - 1)$$

Symbols: n = sample size; k = number of predictors; R^2 = coefficient of determination.

What Does It Measure? It tells you whether added predictors improve the model enough to justify their complexity.

Interpretation:

Value	Meaning
Higher is better	Better balance between fit and complexity.
Can be lower than R^2	Extra variables are not adding value.

Example: If a model has $R^2 = 0.80$, $n = 100$, and $k = 5$, then

$$\text{Adjusted } R^2 = 1 - \left(\frac{1 - 0.80}{100 - 5 - 1}\right)(99) \approx 0.79$$

The adjusted value is slightly lower than the raw R^2, reflecting model complexity.

Python (Using Library):

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
import numpy as np

X = np.array([[1], [2], [3], [4], [5]])
y = np.array([2, 4, 6, 8, 10])
model = LinearRegression().fit(X, y)
print(r2_score(y, model.predict(X)))
```

Python (Without Library):

```python
def adjusted_r2_score_py(y_true, y_pred, k):
    n = len(y_true)
    r2 = 1 - sum((y - p) ** 2 for y, p in zip(y_true, y_pred)) / sum((y - sum(y_true) / n) **
    2 for y in y_true)
    return 1 - ((1 - r2) * (n - 1) / (n - k - 1)) if n - k - 1 > 0 else r2
```

Quick Notes:

- Advantages: better than raw R^2 for comparing models with different feature counts.
- Limitations: still not a universal metric; choose based on the task and data.
- Best use cases: feature selection, multi-variable regression, model comparison.

3.6 MAPE (Mean Absolute Percentage Error)

What is it? MAPE measures prediction error as a percentage of the actual value.

Used For: Regression

When to use: When you want a relative error metric that is easy to explain in percentage terms.

Formula:

$$\text{MAPE} = \frac{100}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right|$$

Symbols: y_i = actual value, $\hat{y}_i$ = predicted value, n = sample count.

What Does It Measure? It measures the average percentage gap between prediction and reality. It is useful for communicating forecast error in familiar percentage terms.

Interpretation:

Value	Meaning
Lower is better	Lower percentage error.
Very high values	Forecasts are often far from actual values.

Example: Actual values: $[100, 200, 300]$, predictions: $[90, 210, 330]$.

$$\left|\frac{100-90}{100}\right| = 0.10, \quad \left|\frac{200-210}{200}\right| = 0.05, \quad \left|\frac{300-330}{300}\right| = 0.10$$

$$\text{MAPE} = \frac{100}{3}(0.10 + 0.05 + 0.10) \approx 8.33\%$$

The average percentage error is about 8.33%.

Python (Using Library):

```python
from sklearn.metrics import mean_absolute_percentage_error

y_true = [100, 200, 300]
y_pred = [90, 210, 330]
print(mean_absolute_percentage_error(y_true, y_pred))
# 0.08333333333333334
```

Python (Without Library):

```python
def mape_py(y_true, y_pred):
    total = 0.0
    for y, p in zip(y_true, y_pred):
        if y == 0:
            continue
        total += abs((y - p) / y)
    return 100 * total / len(y_true)
```

Quick Notes:

- Advantages: easy to communicate as percentages; useful for business forecasting.
- Limitations: breaks down when actual values are near zero; can be skewed by small denominators.
- Best use cases: time-series forecasting, demand prediction, business KPIs.

4. Practical Rule of Thumb

Problem type	Best starting metric
Binary classification, balanced classes	Accuracy
Rare positive events, false negatives costly	Recall, F1, PR-AUC
False positives costly	Precision, specificity
Ranking or probability quality	ROC-AUC, log loss
Regression with interpretable errors	MAE, RMSE
Regression with relative error reporting	MAPE
Comparing model fit while adjusting for complexity	Adjusted R^2

Important note: No single metric is universally best. Choose the metric that matches the consequence of being wrong, then validate with the business or task objective.